ELASTIC LOAD BALANCER (ELB)

AWS Project Documentation

Prepared By: Bashir Shaikh

Key Concepts in Elastic Load Balancing

Elastic Load Balancer (ELB):
An AWS service that automatically distributes incoming application traffic across multiple targets (such as EC2 instances) to ensure high availability, scalability, and fault tolerance. ELB helps balance the load between servers, preventing any single server from becoming overwhelmed.

Security Group:
A virtual firewall in AWS that controls inbound and outbound traffic to your EC2 instances. It acts as a filter for network traffic based on rules defined by the user, ensuring that only authorized traffic can access the instances, enhancing security.

Target Group:
A set of resources (like EC2 instances or containers) that you want the load balancer to route traffic to. Each target group is associated with a specific protocol and port, and it helps the load balancer know where to direct traffic based on the health checks and routing rules you configure.

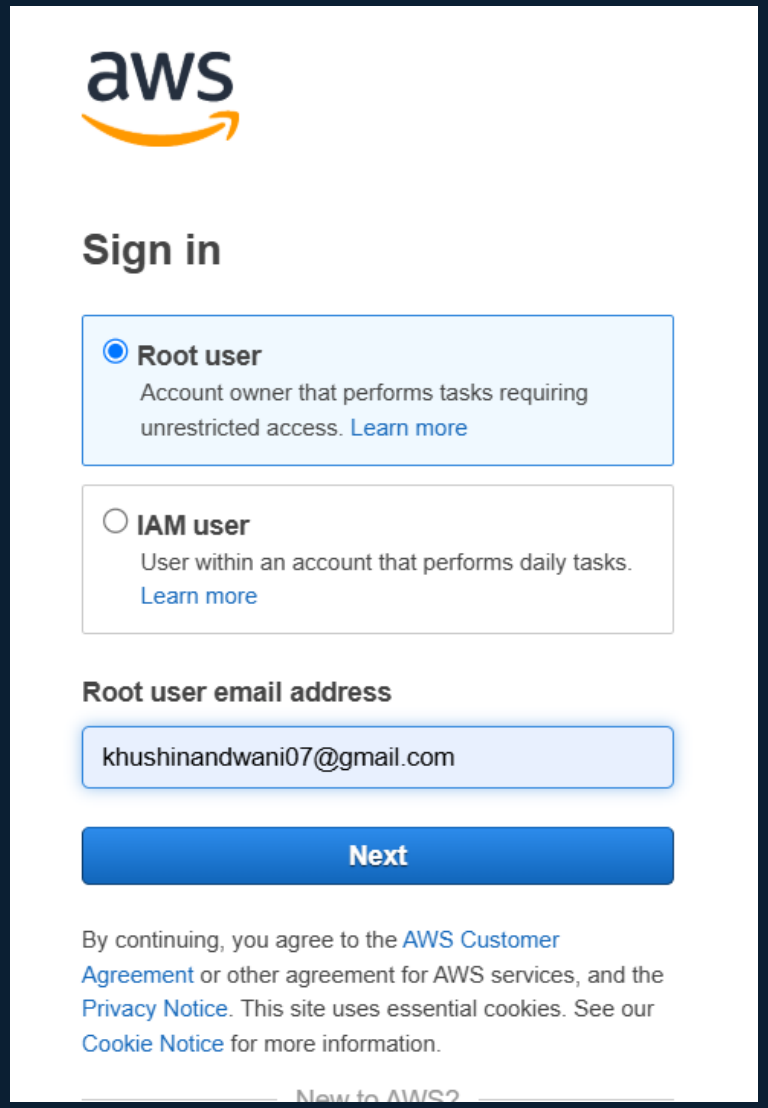
Introduction

This documentation provides a step-by-step guide to creating and launching an AWS ELB setup, configuring security groups, setting up load balancing, and verifying your setup. Let's get started!

Step 1: Log in to AWS Management Console

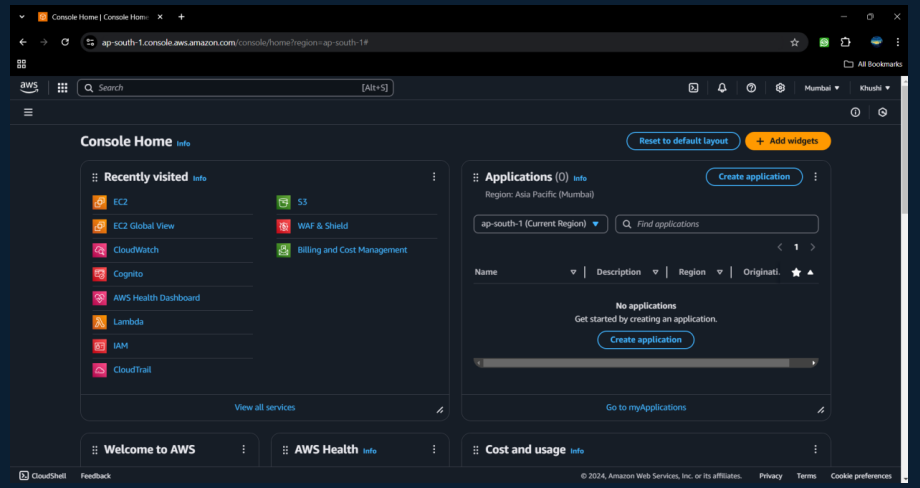
1. Navigate to AWS Management Console.
2. Enter your credentials to log in.

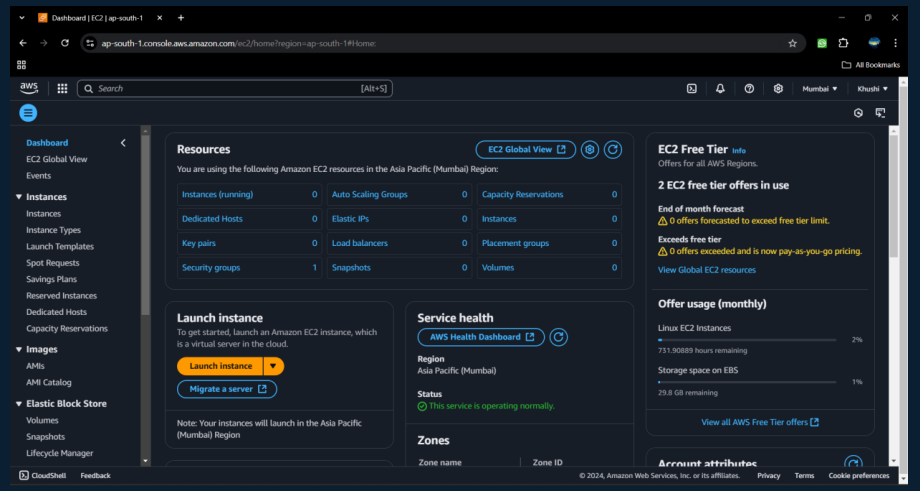
Step 1: Log in to AWS Management Console



Step 2: Search for EC2

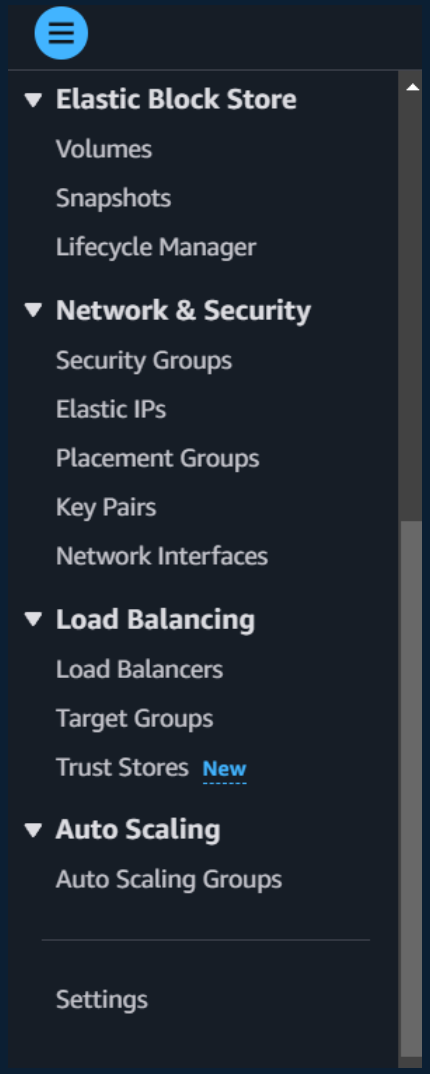
1. In the search bar at the top of the console, type EC2.
2. Click on EC2 to open the EC2 dashboard.





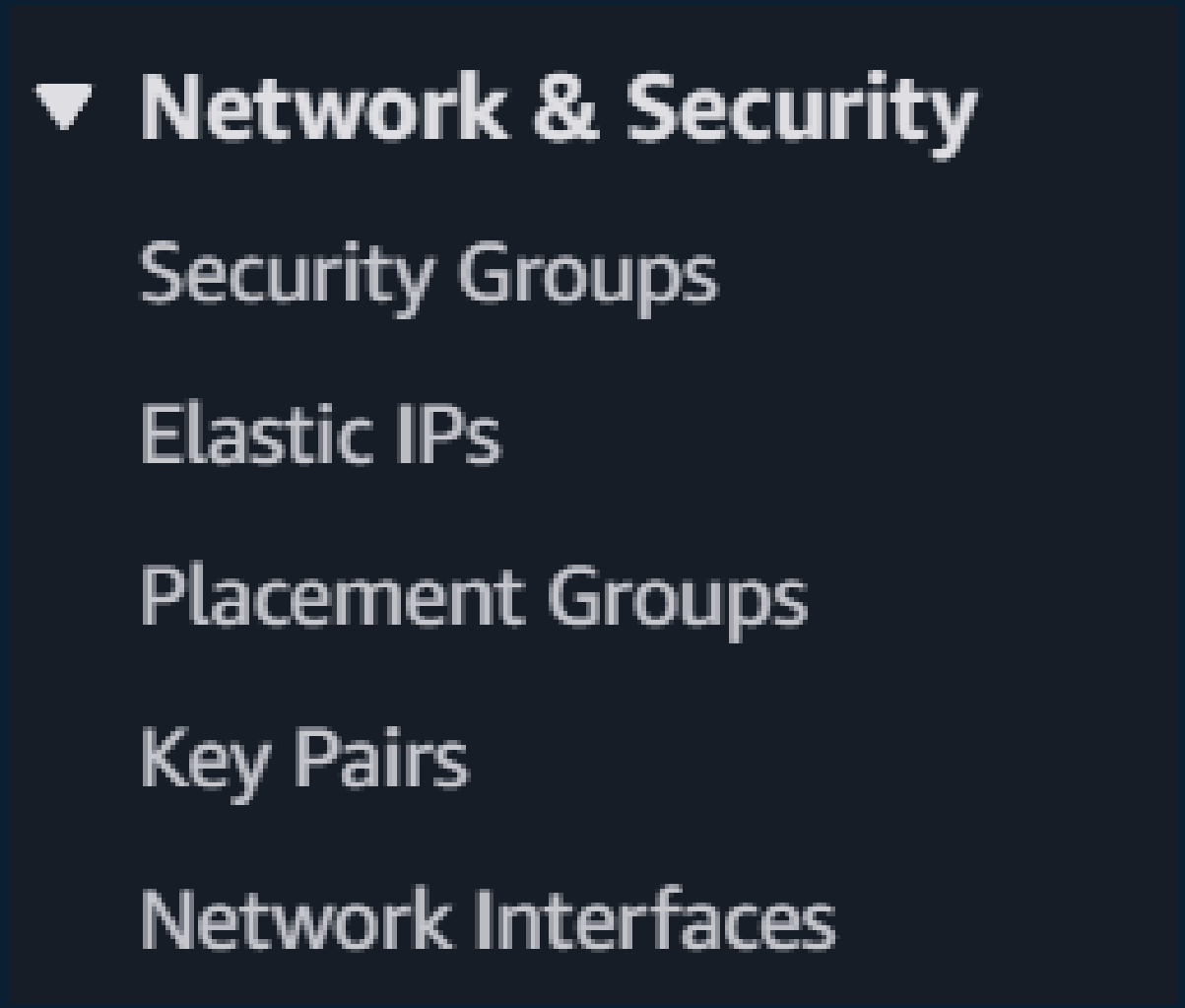
Step 3: Navigate to Security Groups

1. On the EC2 dashboard, scroll to the Network and Security section.



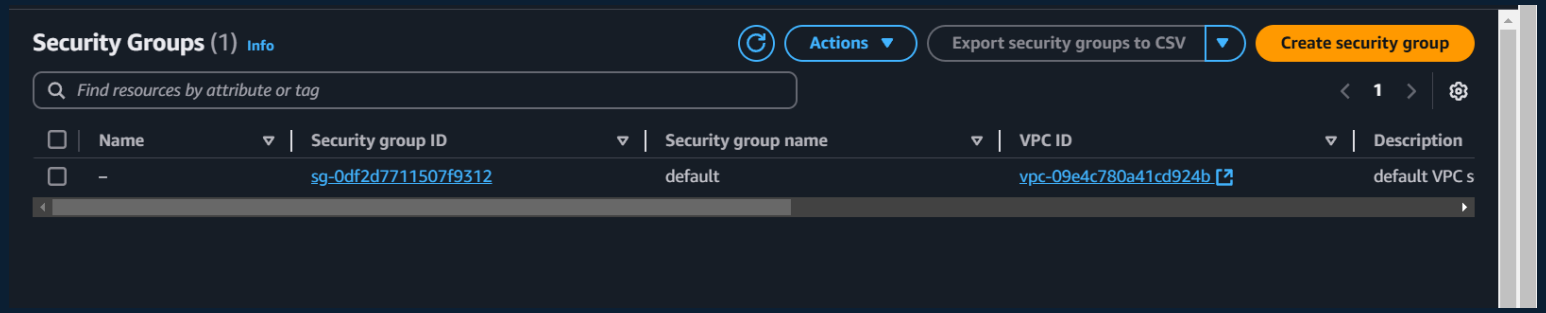
Step 3: Navigate to Security Groups (contd.)

2. Click on Security Groups.



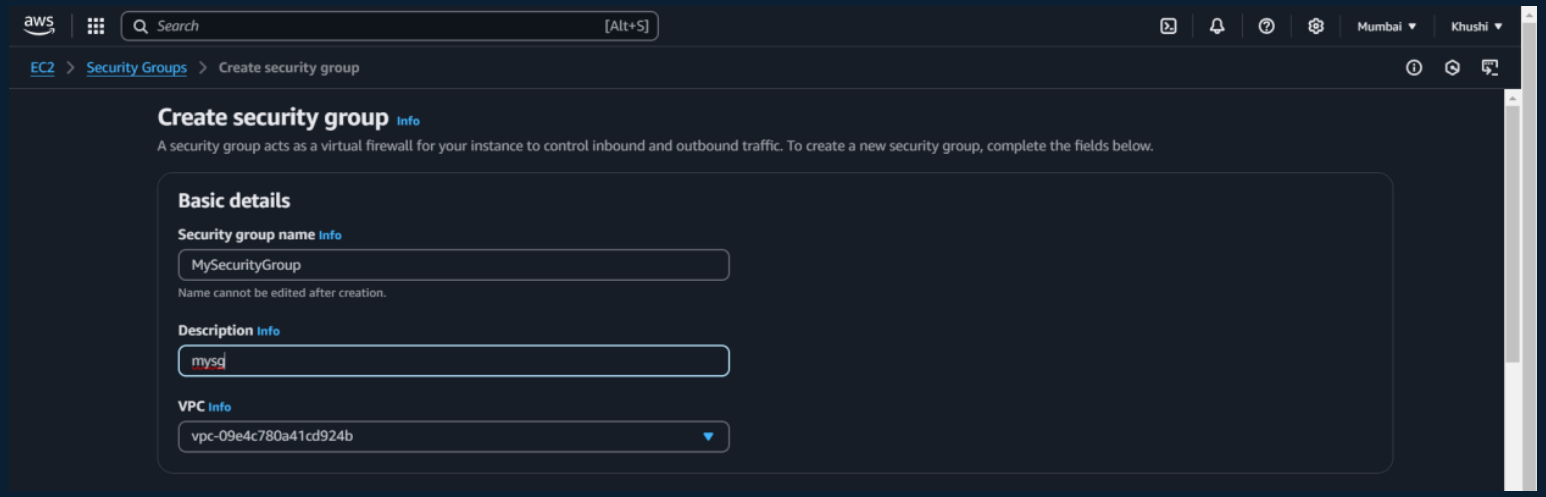
Step 4: Create a Security Group

1. Click on Create Security Group.



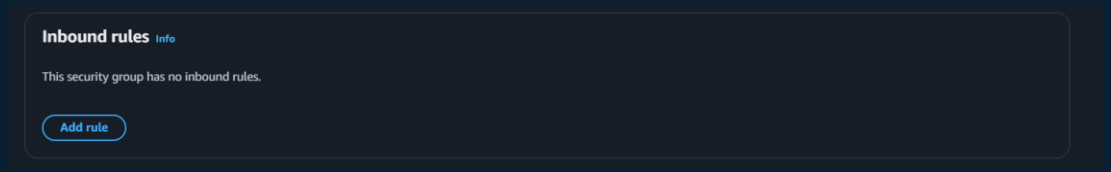
2. Enter the following details:

- Name: MySecurityGroup
- Description: mysg

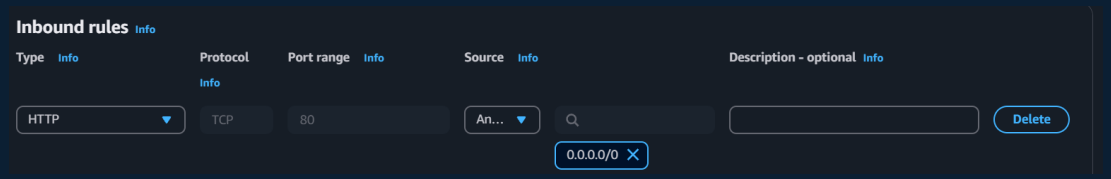


Step 5: Add Inbound Rules

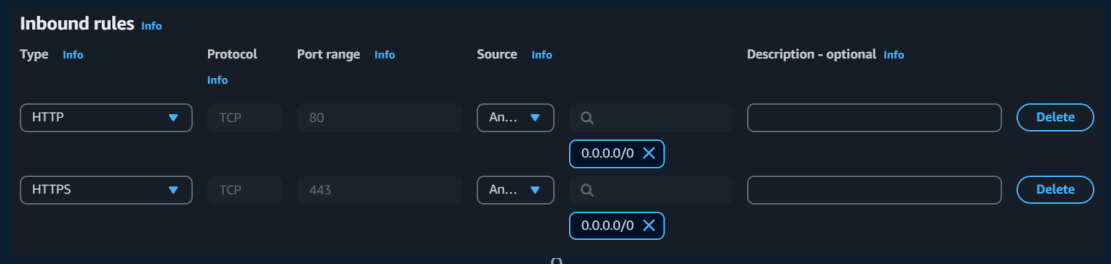
1. Click Add Rules and configure the following:



- Rule 1:
  - Type: HTTP
  - Source Type: Anywhere-IPv4

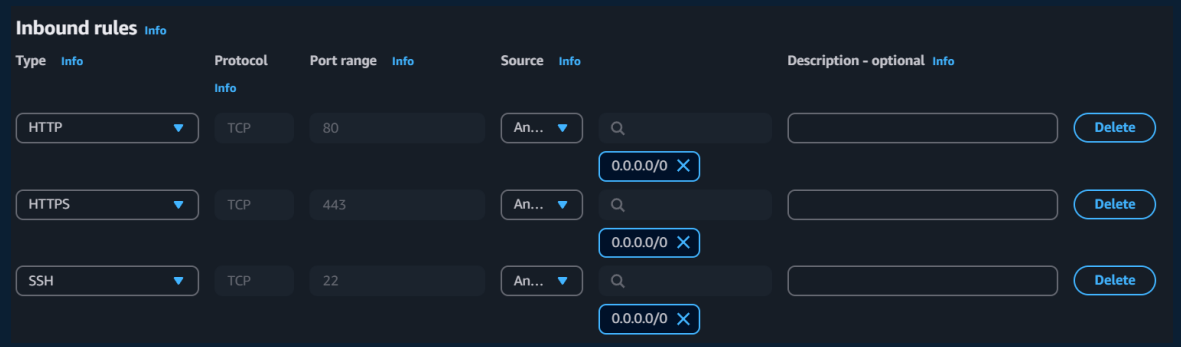


- Rule 2:
  - Type: HTTPS
  - Source Type: Anywhere-IPv4

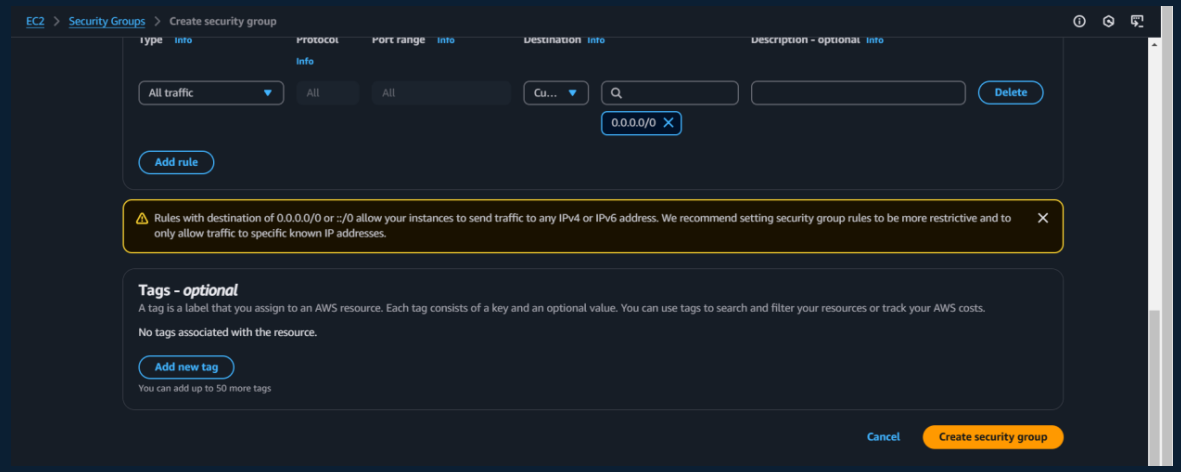


Step 5: Add Inbound Rules (contd.)

- Rule 3:
  - Type: SSH
  - Source Type: Anywhere-IPv4

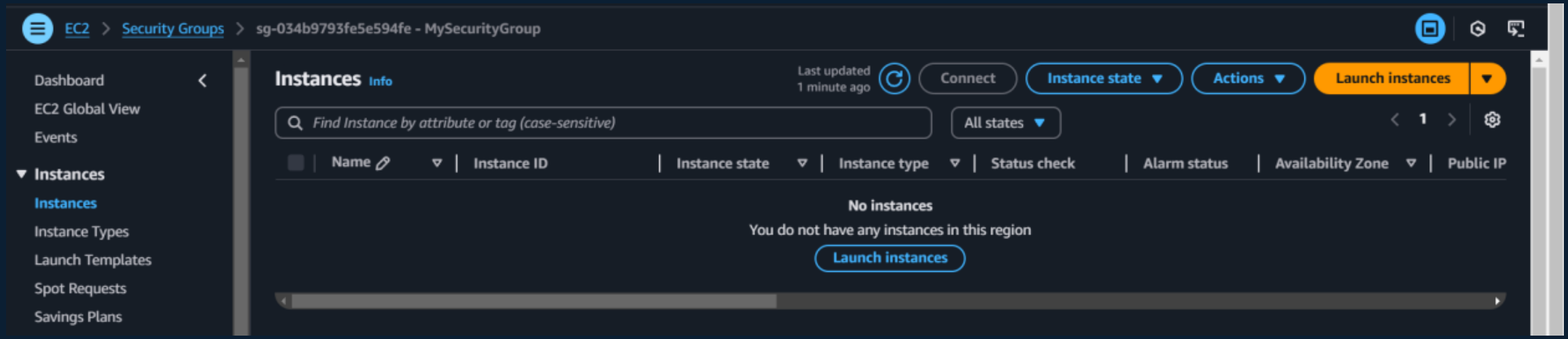


2. Scroll down and click Create Security Group.



Step 6: Launch an EC2 Instance

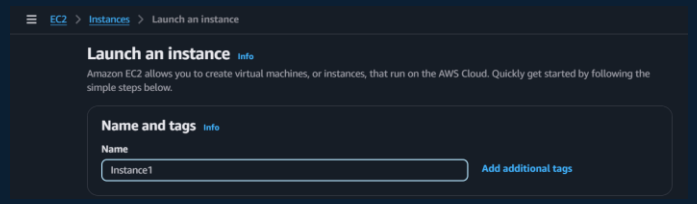
1. Return to the EC2 dashboard and click Launch Instance.



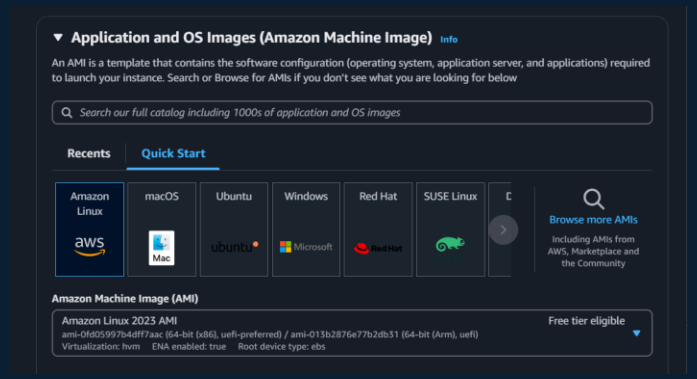
Step 6: Launch an EC2 Instance (contd.)

2. Configure the instance:

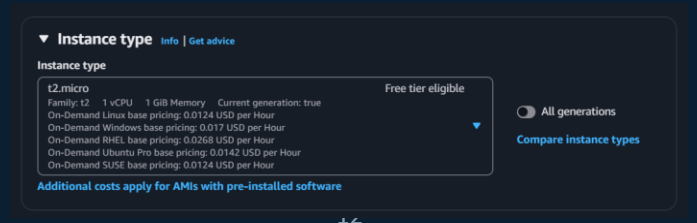
- Name: instance1



- AMI: Amazon Linux

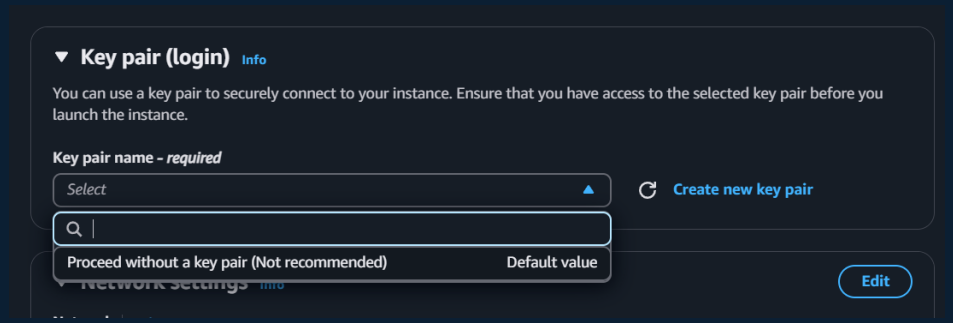


- Instance Type: t2.micro



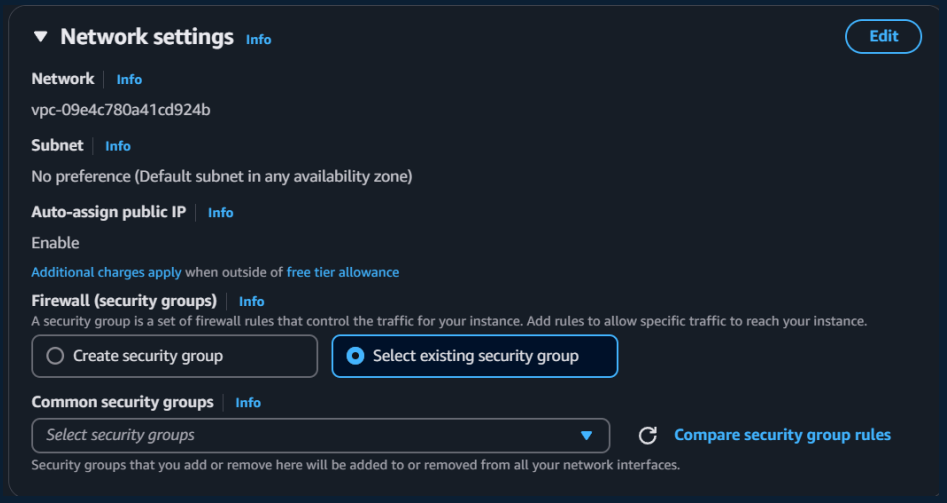
Step 6: Launch an EC2 Instance (contd.)

- Key Pair: Proceed without a key pair.

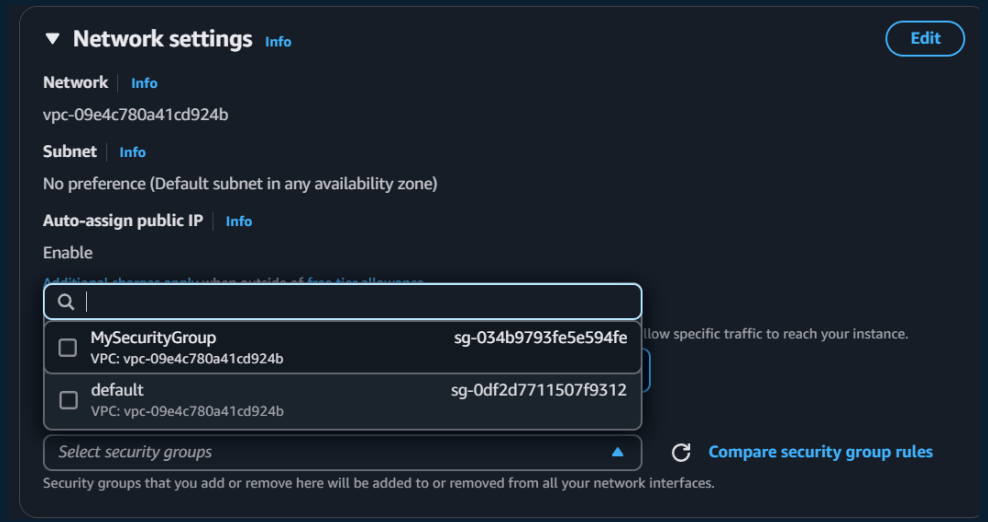


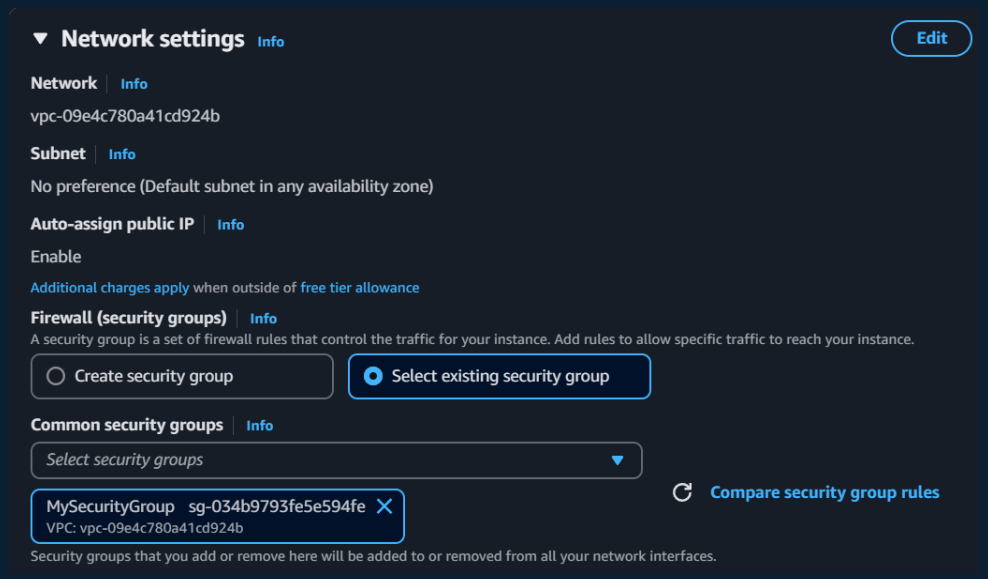
3. Under Network Settings:

- In the Firewall (Security Groups) section, select Existing Security Group and attach MySecurityGroup.



Step 6: Launch an EC2 Instance (contd.)

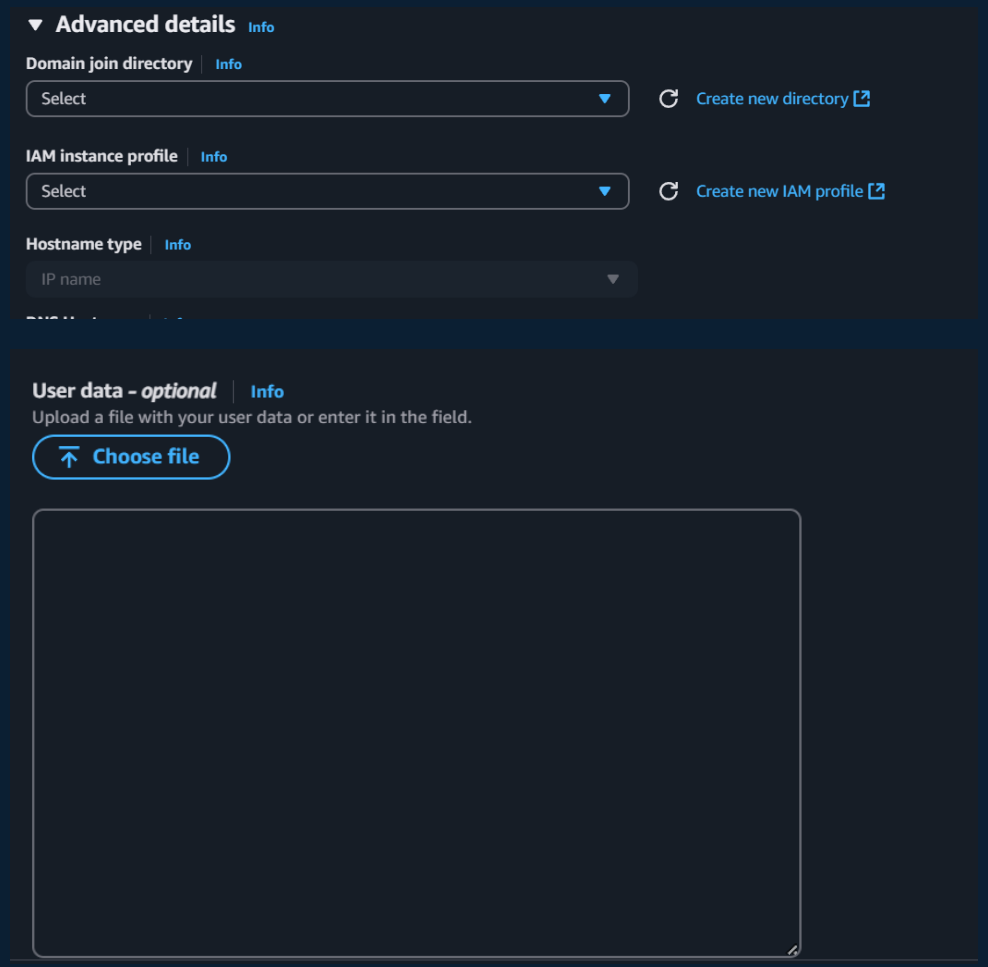




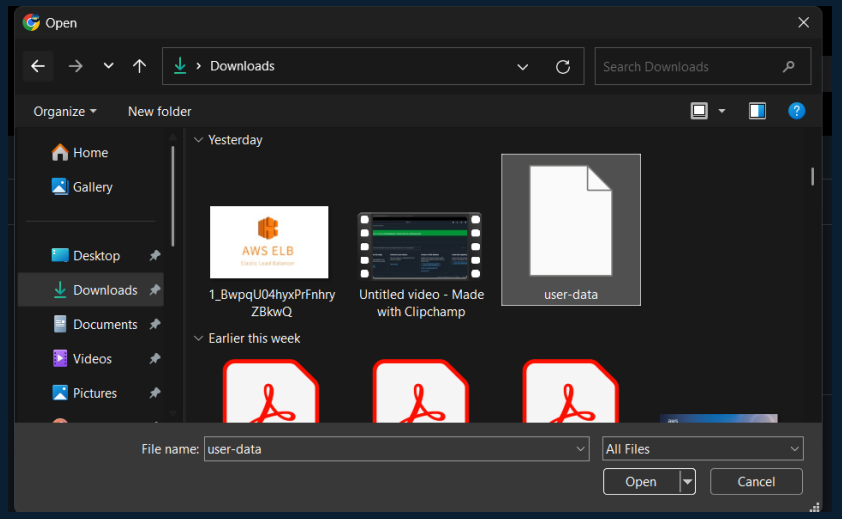
Step 6: Launch an EC2 Instance (contd.)

4. In Advanced Details:

   - Scroll down to User Data and attach your user data file.



Step 6: Launch an EC2 Instance (contd.)



User data - *optional* | Info

Upload a file with your user data or enter it in the field.

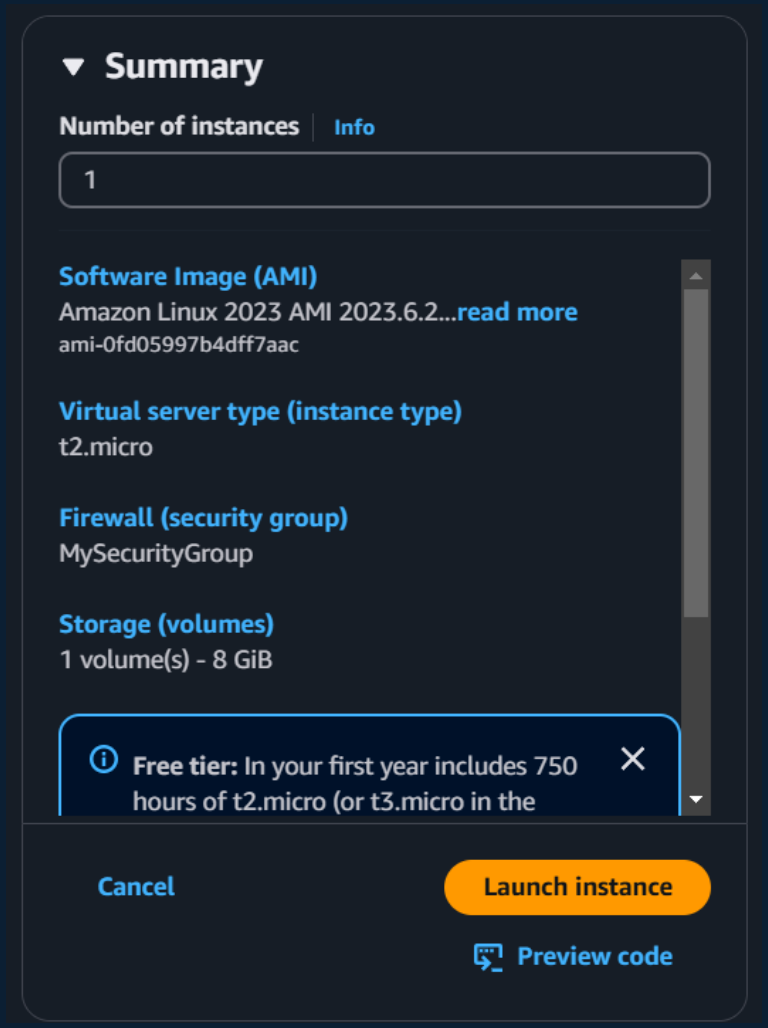
Choose file

```
#!/bin/bash
sudo yum update -y
sudo yum install -y httpd
sudo yum install -y git
export TOKEN=`curl -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600"`
export META_INST_ID=`curl http://169.254.169.254/latest/meta-data/instance-id -H "X-aws-ec2-metadata-token: $TOKEN"`
export META_INST_TYPE=`curl http://169.254.169.254/latest/meta-data/instance-type -H "X-aws-ec2-metadata-token: $TOKEN"`
export META_INST_AZ=`curl http://169.254.169.254/latest/meta-data/placement/availability-zone -H "X-aws-ec2-metadata-token: $TOKEN"`
cd /var/www/html
echo "<!DOCTYPE html>" >> index.html
```

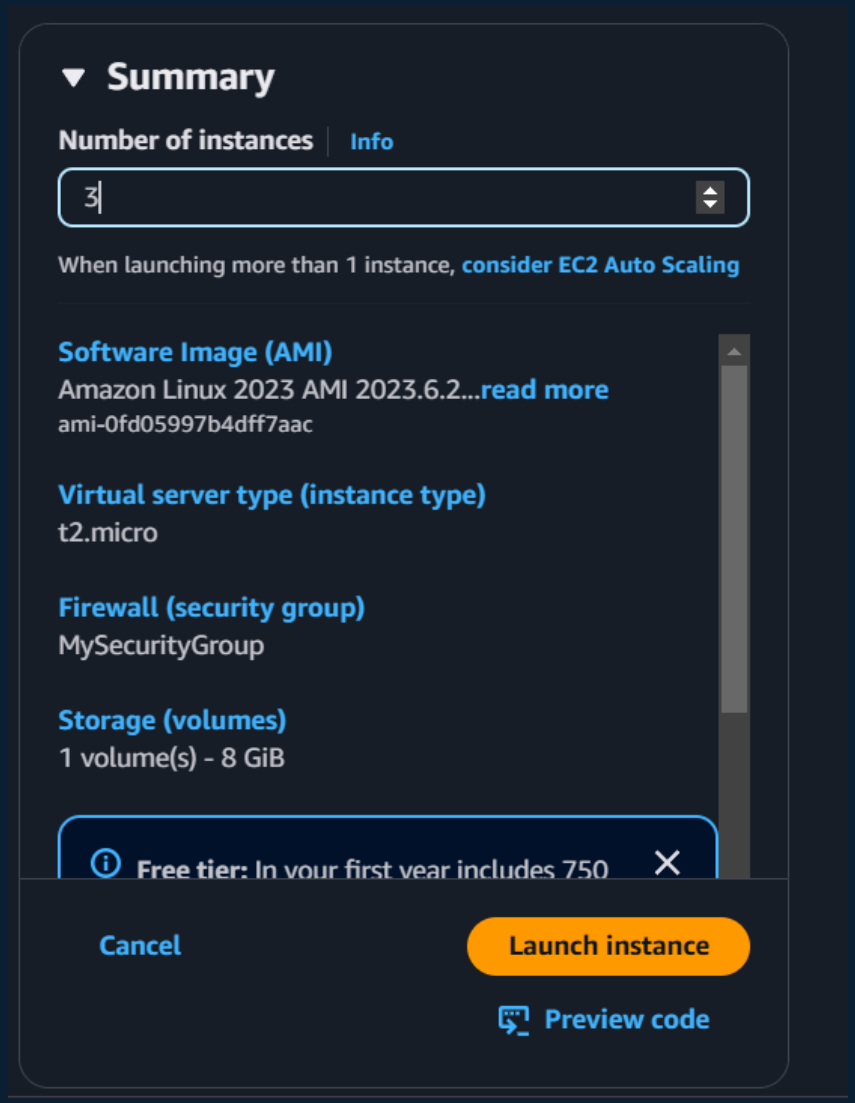
☐ User data has already been base64 encoded

Step 6: Launch an EC2 Instance (contd.)

5. Go to Summary, change the Number of Instances from 1 to 3, and click Launch Instance.

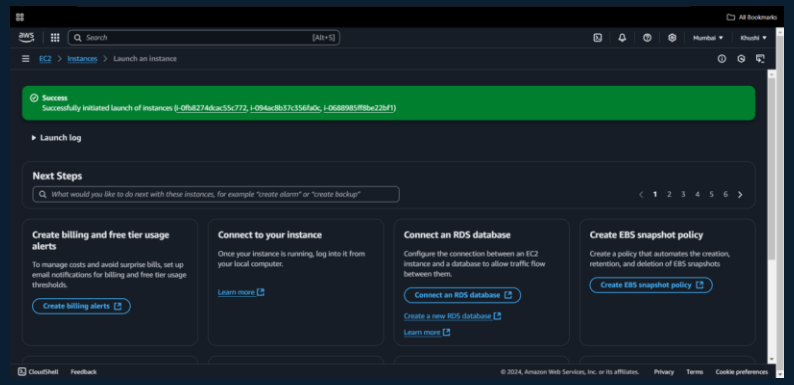


Step 6: Launch an EC2 Instance (contd.)

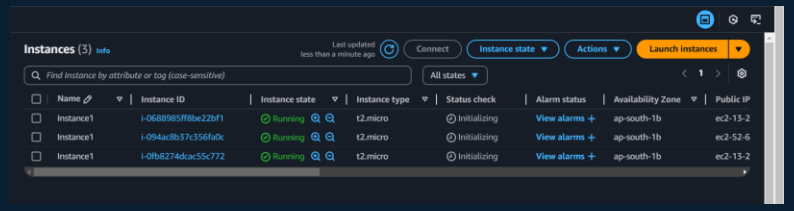


Step 7: Rename Instances

1. Navigate to the View All Instances page.

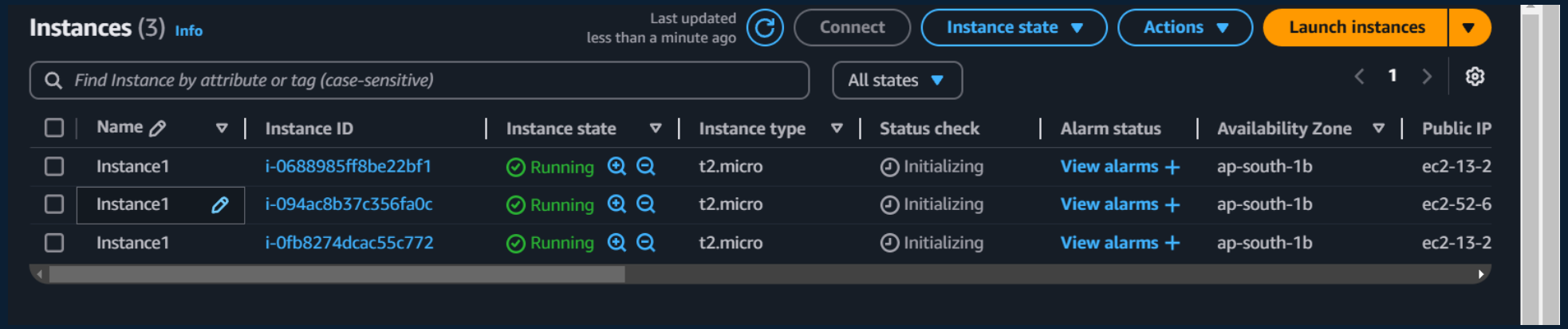


2. Locate your three instances named instance1.

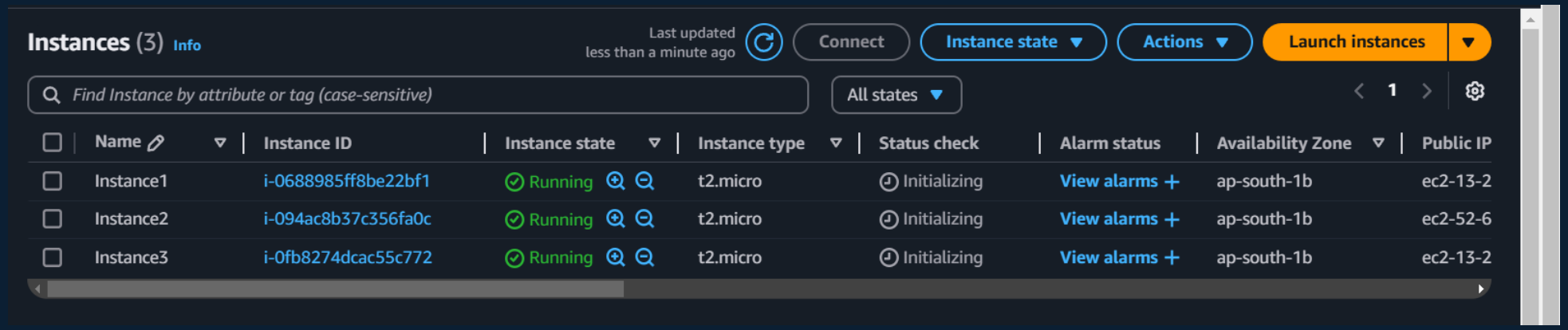


Step 7: Rename Instances (contd.)

3. Hover over each instance name, click the pencil icon, and rename them to:

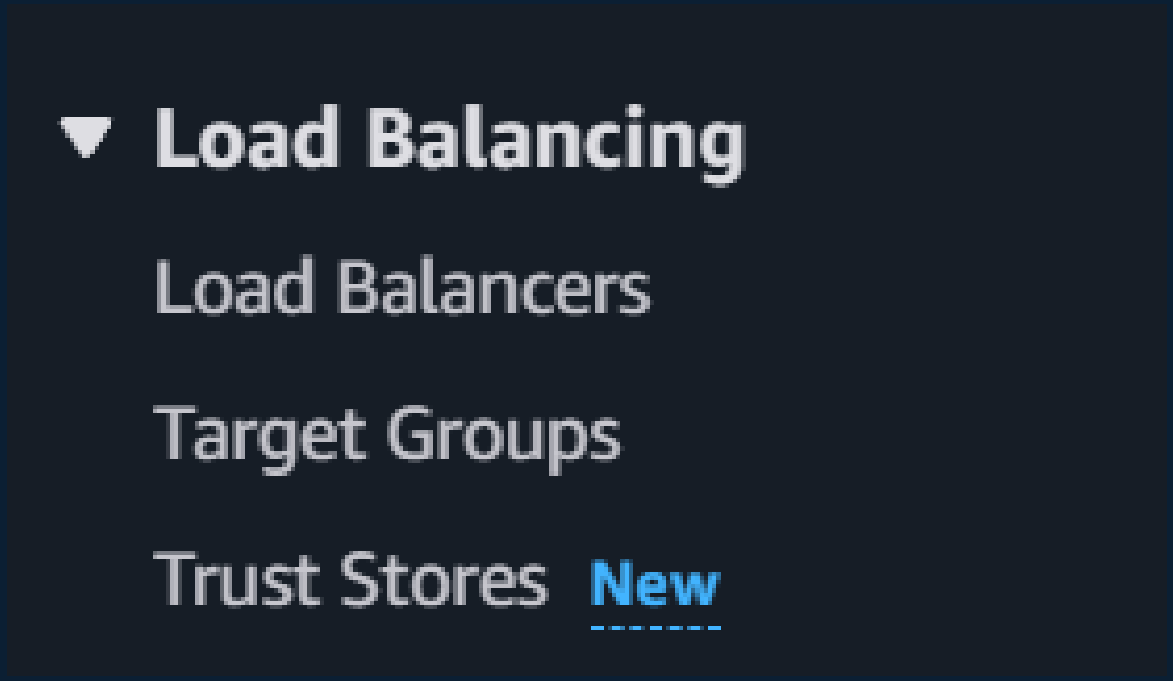


- instance1
- instance2
- instance3

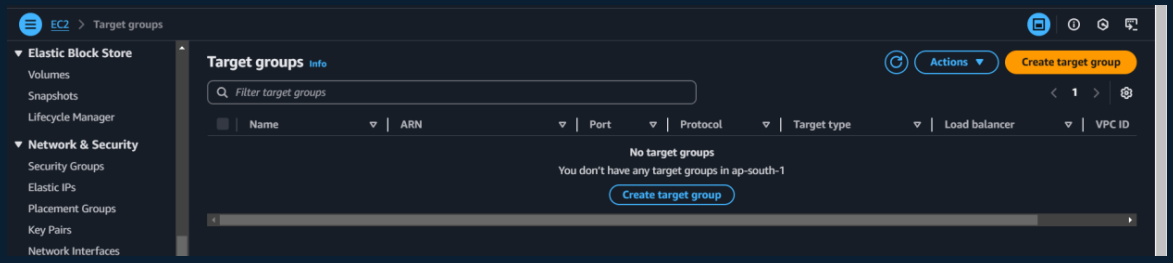


Step 8: Create a Target Group

1. Go to the Load Balancing section and select Target Groups.

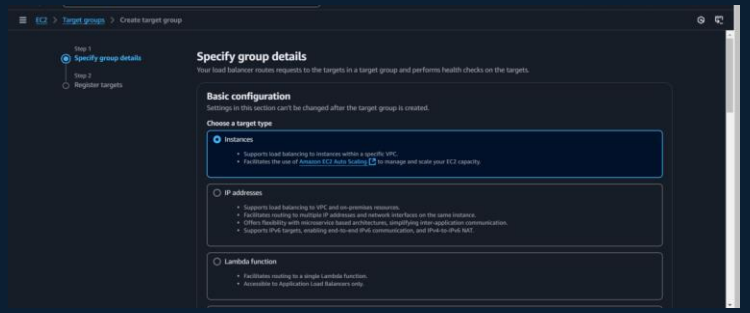


2. Click Create Target Group and configure:

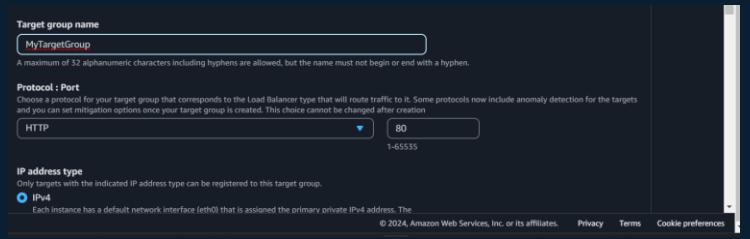


Step 8: Create a Target Group (contd.)

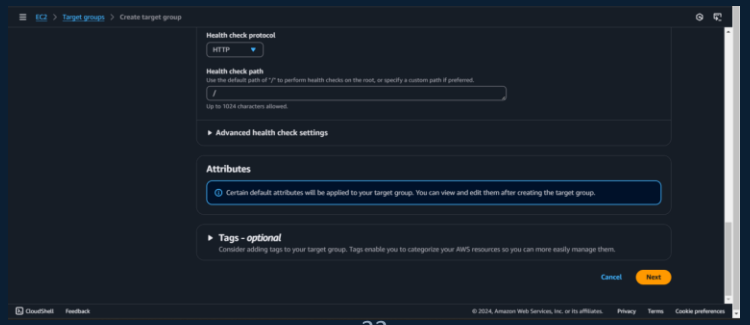
- Type: Instances



- Name: MyTargetGroup

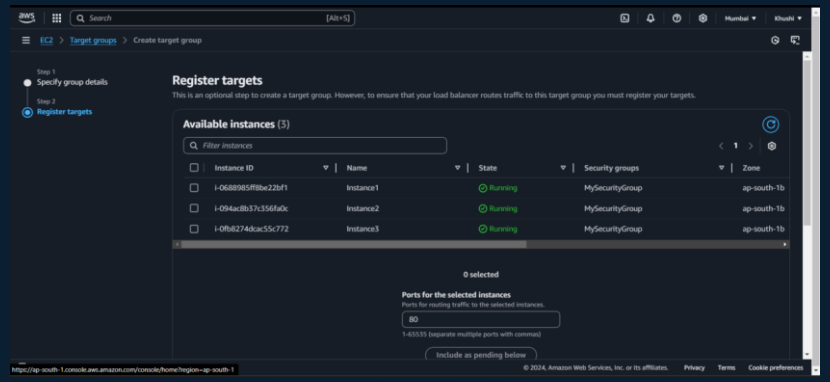


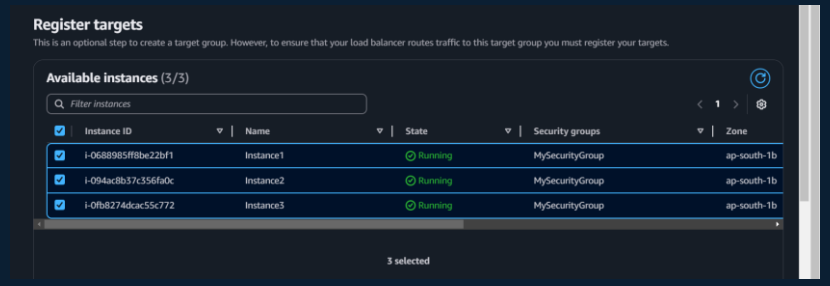
3. Click Next, select all three instances, scroll down, and click Include as Pending Below.

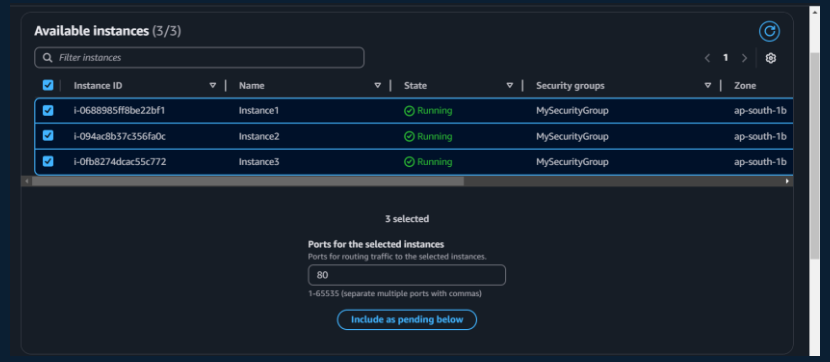


Step 8: Create a Target Group (contd.)

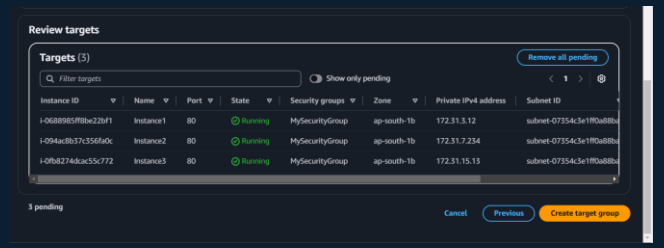
4. Verify the instances in the Review Targets section and click Create Target Group.



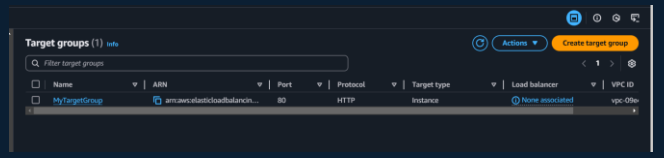




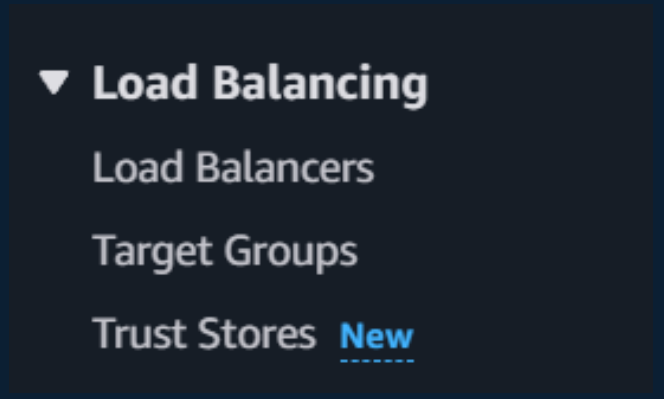
Step 9: Set Up a Load Balancer

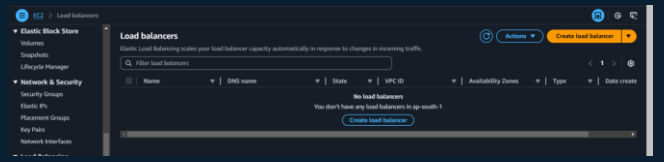


1. In the Load Balancing section, click Load Balancers.



2. Click Create Load Balancer and choose Application Load Balancer.



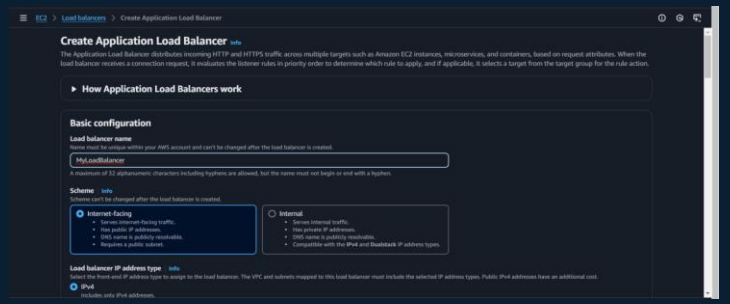


Step 9: Set Up a Load Balancer (contd.)

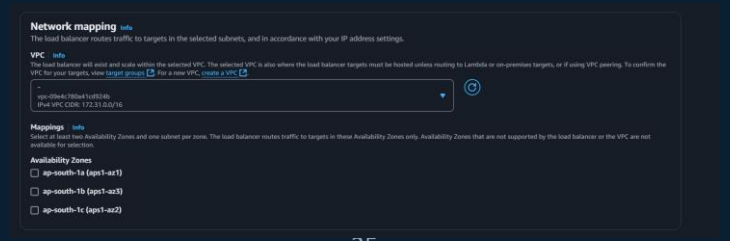


3. Configure the load balancer:

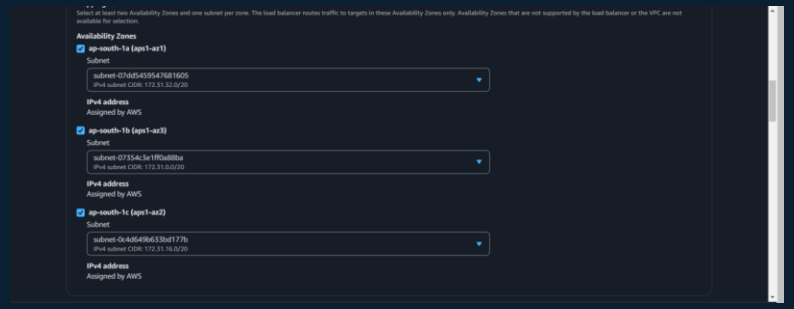
   - Name: MyLoadBalancer



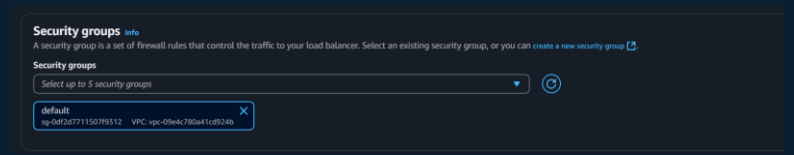
   - Network Mapping: Select all three availability zones.

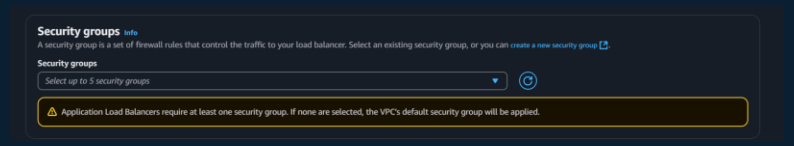


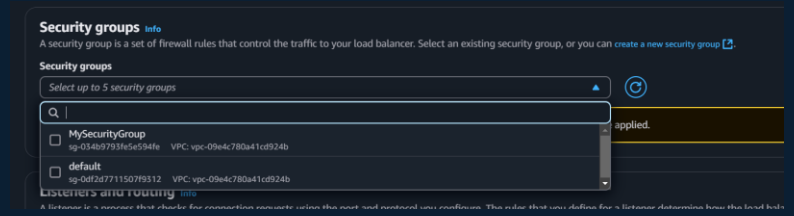
Step 9: Set Up a Load Balancer (contd.)

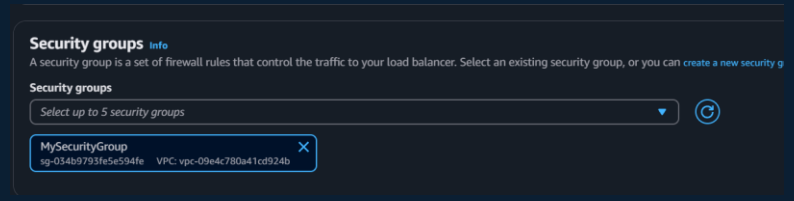


- Security Group: Delete the default group and attach MySecurityGroup.



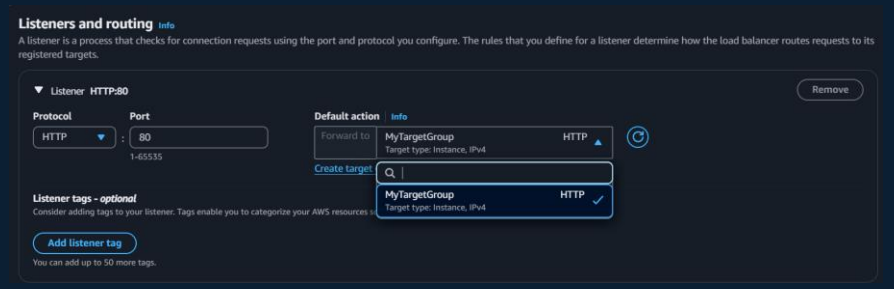


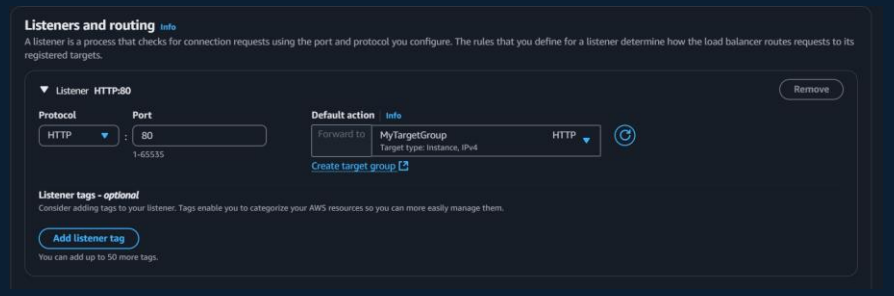




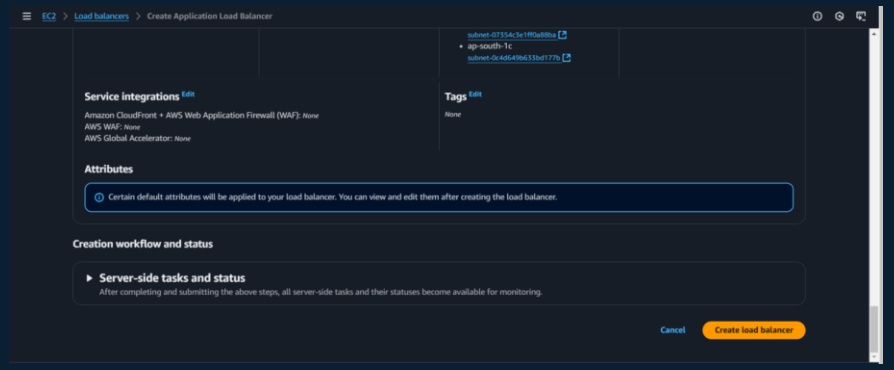
Step 9: Set Up a Load Balancer (contd.)

- Target Group: Select and attach the target group you created.





4. Scroll down and click Create Load Balancer.

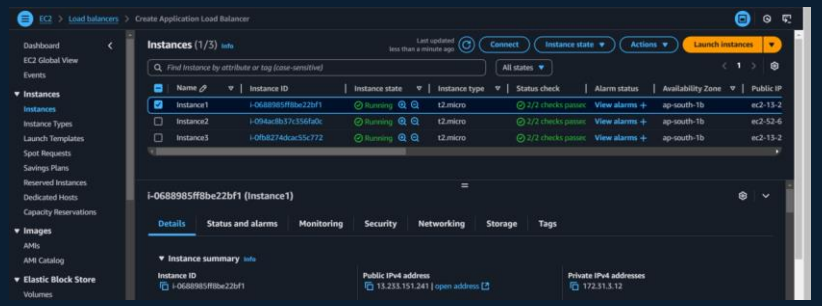


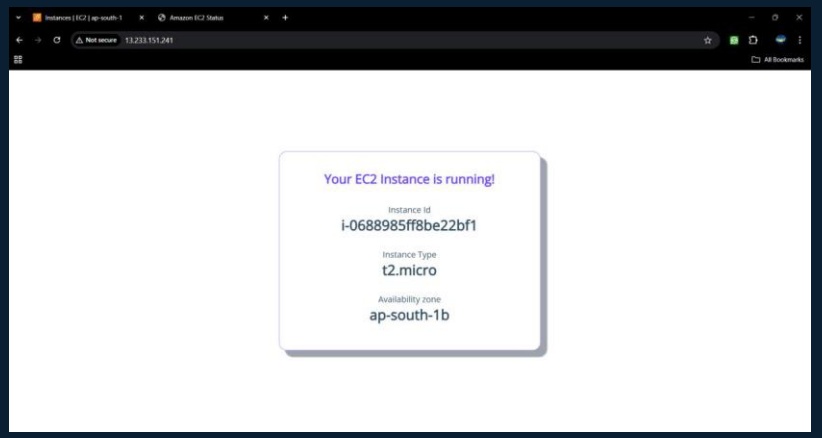
Step 10: Verify Instances and Load Balancer

1. Go back to Instances and check the status of all instances.

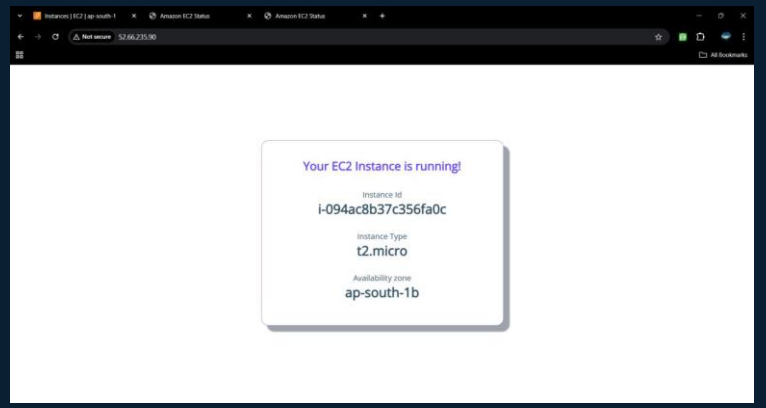


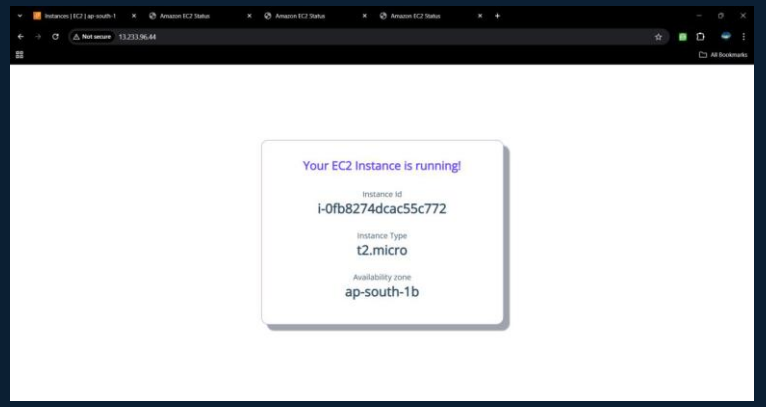
2. Click on each instance and visit their public IP addresses to ensure they are working.



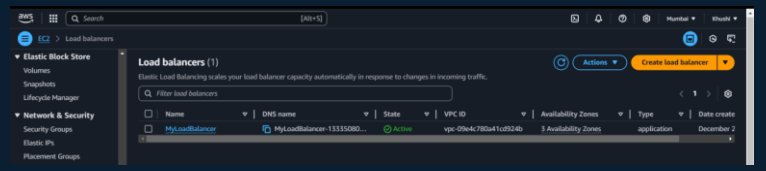


Step 10: Verify Instances and Load Balancer (contd.)

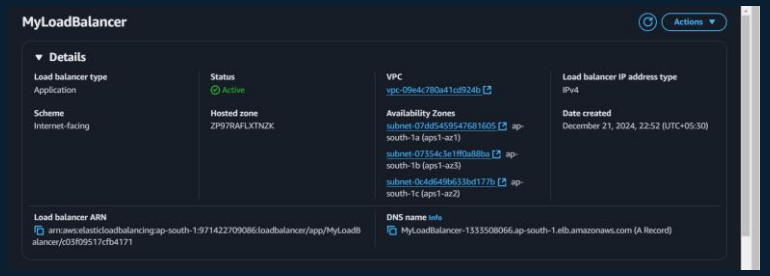




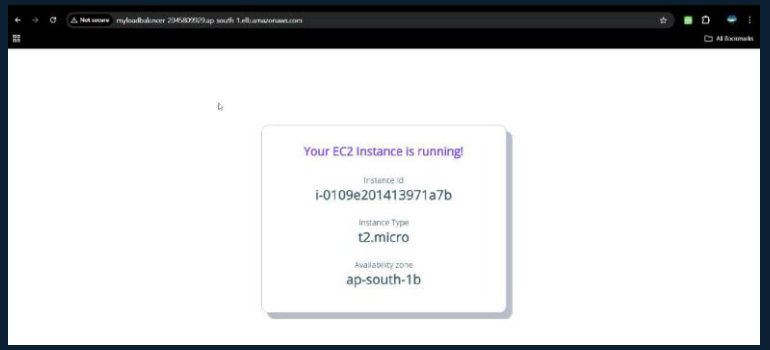
3. Once the load balancer status is Active, click on the load balancer and open the DNS Name in a new tab.

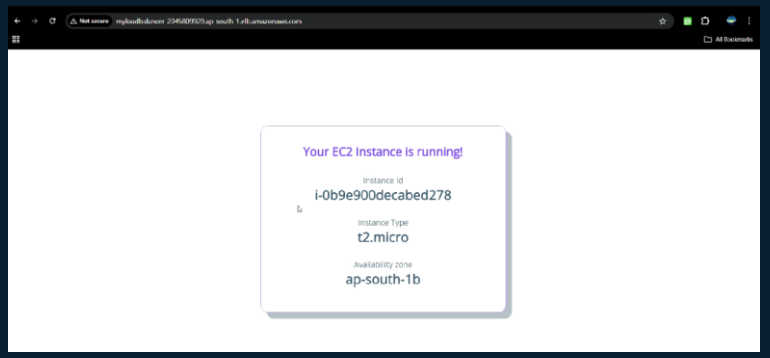


Step 10: Verify Instances and Load Balancer (contd.)



4. Refresh the page to see traffic distributed among different instances, identified by unique instance IDs.

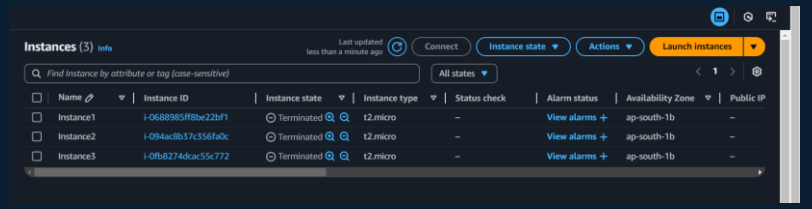




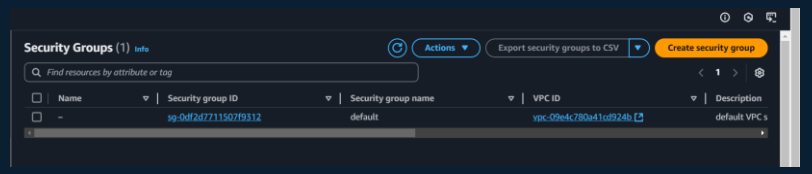
Step 11: Clean Up

To avoid unnecessary charges, ensure you:

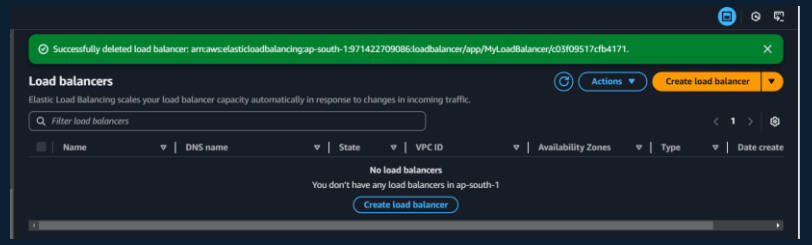
1. Terminate all instances.

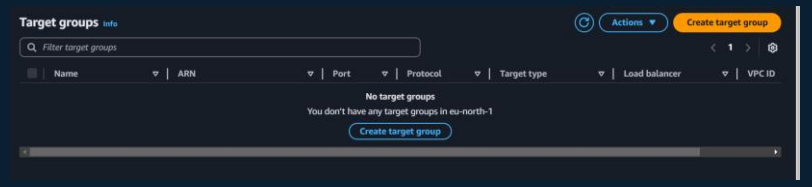


2. Delete the security group.



3. Remove the load balancer and target group.





Conclusion

In this project, we successfully set up an AWS Elastic Load Balancer (ELB) environment to ensure high availability and scalability for our application. The process involved creating and attaching a security group to enhance security, launching multiple instances for load balancing, and configuring the ELB to efficiently distribute traffic across the instances. This setup ensures optimal performance, fault tolerance, and improved user experience by balancing the load and preventing server overload.

With this implementation, the system is now capable of handling increased traffic while maintaining stability and availability.

Moving forward, this setup can be easily scaled to accommodate future growth, ensuring the system remains robust and reliable under varying loads.